تقرير أنماط التصميم البرمجية المطبقة في منصة VendorProof

دليل شامل للإجابة الأكاديمية والمناقشة الجامعية (Software Design Patterns Report)

📌 مقدمة تمهيدية للمناقشة

عند سؤال الدكتور أو لجنة التحكيم:

> **"استخدمت في المشروع؟ وأين هي؟ وكيف تعمل؟ ولماذا اخترتها؟ (Design Pattern) كم نمط تصميم"**

الإجابة النموذجية المباشرة:

> "اعتمدنا في المنصة على مجموعة من **أنماط التصميم المعمارية والسلوكية المعترف بها عالمياً** لضمان عزل المنطق **(Enterprise Design Patterns)**. ومن أهم هذه الأنماط، اخترنا **ثلاثة أنماط جوهرية** تدير عصب (Scalability) وقابلية التوسع (Multi-Tenancy Security)، وحماية بيانات المنشآت (Decoupling)، النظام، بالإضافة إلى الأنماط الهيكلية المدمجة في إطار العمل."

🏆 أفضل 3 أنماط تصميم رئيسية مطبقة فعلياً في المشروع

النمط الأول: نمط طبقة الخدمة (Service Layer Pattern)

ينتمي إلى: أنماط المعمارية البرمجية للمؤسسات (Enterprise Architectural Patterns)

1. أين يوجد في الكود؟ (Where)

- ملف خدمة المنشآت: `app/Services/Tenants/TenantService.php`
- ملف خدمة الباقات: `app/Modules/Web/Services/Plans/PlanService.php`

2. كيف تم تطبيقه؟ (How)

تم عزل العمليات التجارية المعقدة (Business Logic) وحركات قواعد البيانات المركبة داخل كلاسات خدمة مستقلة بدلاً من حشوها داخل المتحكمات (Controllers).

- **مثال حي من المشروع (`TenantService.php`)**: عند تسجيل منشأة جديدة، يتم استدعاء دالة `createTenant()` التي تقوم **بشكل ذري (Atomic Transaction)** بتنفيذ سلسلة عمليات متكاملة داخل `DB::transaction`:
  1. إنشاء سجل المنشأة في جدول `tenants`.
  2. إنشاء حساب المدير التنفيذي وتوليد كلمة مروره المشفرة في جدول `users`.
  3. ربط المنشأة بالباقة المختارة وإنشاء سجل اشتراك رسمي في جدول `subscriptions`.
  4. في حال حدوث أي خطأ في أي خطوة، يتم عمل تراجع تلقائي (`Rollback`) لحماية سلامة البيانات.

```php
// استدعاء نظيف من المتحكم في سطر واحد دون تعقيد:
$result = $tenantService->createTenant($tenantData, $userData, $freePlan->id);
```

3. لماذا استخدمناه؟ (Why)

- **مبدأ المسؤولية الواحدة (Single Responsibility Principle - SRP)**: أصبح دور المتحكم محصوراً في استقبال الطلب وإرجاع الرد، بينما منطق الأعمال يعيش في الخدمة.
- **إعادة الاستخدام (DRY - Don't Repeat Yourself)**: نفس دالة التأسيس `createTenant` يتم استدعاؤها من واجهة الويب الإدارية ومن الـ API العام دون تكرار سطر كود واحد.

- **سهولة الاختبار الآلي (Unit & Feature Testing)**: إمكانية اختبار منطق تأسيس الشركات بمعزل تام عن مسارات الـ HTTP.

النمط الثاني: نمط نطاق الاستعلام والاعتراض التلقائي (Global Scope / Interceptor Pattern)

ينتمي إلى: الأنماط السلوكية والاعتراضية (Behavioral / Query Interceptor Patterns)

1. أين يوجد في الكود؟ (Where)

- ملف سكوب العزل: `app/Models/Scopes/TenantScope.php`
- ملف السمة المعمارية: `app/Traits/BelongsToTenant.php`

2. كيف تم تطبيقه؟ (How)

يجب ضمان عدم تسرب بيانات شركة إلى شركة أخرى، متعددة المنشآت (Multi-Tenant) في منصات الـ SaaS.

- تم بناء كلاس `TenantScope` يطبق واجهة `Scope` في لارافيل ليعترض دورة حياة أي استعلام (Query Lifecycle).
- عند ربط السمة `BelongsToTenant` بأي موديل، يقوم السكوب تلقائياً وبشكل غير مرئي بحقن شرط الأمان:

```
SELECT * FROM users WHERE tenant_id = 'current-tenant-uuid';
```

- وعند إنشاء أي سجل جديد، تلتقط السمة حدث `creating` وتقوم تلقائياً بتعبئة حقل `tenant_id` بهوية المنشأة الحالية الموثقة دون تدخل يدوي من المطور.

3. لماذا استخدمناه؟ (Why)

- **منع كارثة تسرب البيانات (Zero Data Leakage Guarantee)**: حماية قصوى للنظام؛ حتى لو نسي المبرمج كتابة `where('tenant_id', ...)` في استعلاماته، فإن نمط السكوب يفرض الشرط إجبارياً على مستوى قاعدة البيانات.
- **الشفافية البرمجية (Transparency)**: كتابة استعلامات نظيفة مثل `User::all()` مع الاطمئنان التام بأنها لن تُرجع إلا مستخدمي المنشأة صاحبة الجلسة فقط.

النمط الثالث: نمط الكائن المشرف الأساسي وعقد الواجهات الموحد (Layer Supertype & Template Response Pattern)

ينتمي إلى: أنماط التصميم الهيكلية والقوالب المعيارية (Structural & Template Method Patterns)

1. أين يوجد في الكود؟ (Where)

- متحكم الـ API الأساسي: `app/Modules/Api/Http/Controllers/ApiController.php`
- ومتحكمات الـ API المشتقة منه: `ApiAuthController.php` و `ApiTenantRegistrationController.php`

2. كيف تم تطبيقه؟ (How)

تم إنشاء كلاس تجريدي فائق للطبقة (Layer Supertype) يحمل عقد الاستجابة المعياري الموحد للمشروع:

- يحتوي الكلاس على دالتين مركزيتين:
  - `successResponse($data, $message, $status)`
  - `errorResponse($message, $status, $errors)`
- يدوياً أبداً، مما يجبر كافة الاستجابات على الخروج بهيكل `response()->json([...])` من هذا الكلاس حصرياً، ومحكومة بعدم كتابة ترث كافة متحكمات الـ API بيانات موحد:

```
{
  "success": true,
  "message": "تمت العملية بنجاح",
  "data": { ... }
}
```

3. لماذا استخدمناه؟ (Why)

- **توحيد عقد الربط مع الفرونت إند (API Contract Standardization)**: يتعامل مع صيغة رد واحدة وثابتة في كامل مطور الواجهات الأمامية (Frontend Engineer) المنصة، مما يمكنه من كتابة `Interceptor` واحد في الـ Frontend لمعالجة النجاح والخطأ بكفاءة.
- **المركزية وقابلية التوسع (Centralized Governance)**: نعدل في سطر واحد داخل API، لجميع استجابات الـ Logging إذا أردنا إضافة معالجة عامة أو تشفير أو `ApiController` فينعكس فوراً على كامل النظام.

🎖️ أنماط تصميم إضافية مطبقة في المشروع (لتعزيز قوة الإجابة)

إذا أراد الدكتور المزيد أو سألك: "*هل توجد أنماط أخرى؟*" يمكنك ذكر ما يلي بثقة:

1. **نمط التجزئة الرأسية الموديلية (Vertical Slice / Modular Architecture Pattern)**:
   - تم تطبيقه بفصل المشروع إلى موديولين رأسيين: `app/Modules/Api/` (لمحرك الأداة والمستخدمين) و `app/Modules/Web/` (لإدارة المنصة والسوبر أدمن)، مما منع التضارب والفوضى (والتوكنات).
2. **نمط السجل النشط (Active Record Pattern)**:
   - حيث يمثل كل كائن سطراً في قاعدة البيانات مع دوال التحكم (`Admin` , `Tenant` , `Plan` , `User`) وهو النمط الأساسي المعتمد في موديلات Eloquent بعلاقاته.
3. **نمط حقن التبعيات ومقلوب التحكم (Dependency Injection & Inversion of Control - IoC)**:
   - تم تطبيقه في دوال المتحكمات مثل `register(ApiRegisterTenantRequest $request, TenantService $tenantService)` حيث يقوم محرك لارافيل بحقن الخدمات آلياً دون الحاجة لكتابة `new TenantService()` .
4. **نمط مواصفات قيود الميزات (Specification / Feature Gating Pattern)**:
   - مطبق في سمة `HasPlanFeatures.php` لفحص حدود واشتراكات المنشآت (`canAccessFeature` , `dynamicLimits` , `lockedFeatureKeys`) ومنع استهلاك ميزات غير مشمولة في الباقة.

📊 جدول ملخص سريع للمناقشة والمراجعة السريعة

نمط التصميم (Pattern)	تصنيفه العالمي	موقعه في مشروعك	الفائدة التقنية المباشرة
Service Layer	Enterprise Pattern	`TenantService.php`	عزل العمليات التجارية وضمان الحركات الذرية `DB::transaction` .
Global Scope / Interceptor	Behavioral / Query Pattern	`TenantScope.php`	العزل الأمني التلقائي لبيانات المنشآت ومنع التسريب (Multi-Tenancy).
Layer Supertype	Structural / Template Pattern	`ApiController.php`	توحيد شكل استجابات الـ JSON وتسهيل تكامل الواجهات الأمامية.
Modular Architecture	Architectural Pattern	`app/Modules/Web` & `Api`	الفصل التام بين لوحة المشرف ومحرك الأداة الموجه للفرونت الخارجي.
Dependency Injection	Creational / IoC Pattern	`Controllers` `Constructor/Methods`	فك الارتباط المباشر وتسهيل عمل الاختبارات الآلية (Mocking).

> **نصيحة ذهبية عند المناقشة:** ركّز للدكتور على أن هذه الأنماط **لم تُكتب كاستعراض نظري**، بل حلت مشاكل حقيقية في النظام: عزل المنشآت لحماية الأمان (Security)، وضمان تكامل المعاملات المالية والاشتراكات بدون أخطاء (Data Integrity)، وتوحيد الـ API لراحة مطور الفرونت (Productivity).